Homework 2

Lucas Sarweh

110042658

COMP-3300

Due: February 6th, 2024, 10AM

1. Batch: A system that carries out jobs in batches. These jobs may be scheduled at certain times. Used when jobs can be run automatically, and resources are idle to use as a user cannot keep idle resources busy. Job scheduling determines what to run next.

   Interactive: A system that uses inputs from users. The main purpose is to serve the user of the system for ease of use.

   Time-slicing: System with many users. The CPU is shared among the users by switching jobs frequently. Users interact with each job. Emphasis on response time over CPU use. Appropriate when multiple user interaction is needed.

   Real-time: A system that has a special computing environment done in real-time. Has very strictly defined fixed time constraints. Must process within constraints or operations are incorrect. Appropriate when time sensitive computation is required.

   Distributed: A system where multiple computer systems interact with each other in a network. Gives the illusion of a single system. TCP/IP most common. Good use of divide and conquer.

2. A program is a set of computer instructions that is not being used. An executable is a program that can be directly run by the OS or hardware. A process is a program that is currently being run by the computer using CPU, I/O, memory and other resources.

3. This could be implemented by calculating the time to wake up the process, then placing the process/thread into a pending queue so that the job won't be scheduled. Then place the job back into the ready queue after the time calculated is reached.

4. A context switch saves a process that was running and loads a new process. The PCB state is saved which includes info such as the program counter, registers, I/O status, memory management info, accounting, scheduling info and more. Overhead depends on hardware support. This should happen frequently because the schedulers will give the CPU to different processes multiple times. In order to keep the CPU busy and give a share of the CPU to each program in the ready queue.

5. No because the process id must be specified in those send and receive calls. It must know all pids.

```
message msg;
While(true){
        for i = 0 to n{
                if empty(i){
                        receive(i, msg);
                }
        }
}
```

6.
   a. The number of processors is more than the number of kernel threads so each kernel thread can have its own processor. However, this makes some of the processors idle. The number of user threads is more than the kernel threads so there is a bottleneck because there are less kernel threads.
   b. The number of kernel threads is equal to processors, so each kernel thread has its own processor, and no processor is left idle. However, the kernel threads are still fewer than the user threads, so it is still bottlenecked like in a one-to-many threading when blocking occurs.
   c. There are one or more kernel threads per processor, so no processor is idle. Because of this there are multiple threads in a processor, so you have a real many-to-many threading relationship. Many kernel threads for the user threads have the greatest performance of a, b, and c.